

Claude Code

A Practical Study Deck - 16 Topics

Original study notes

Introduction → Wrapping Up

01 · Introduction to Claude Code

Claude Code is Anthropic's terminal-first agentic coding tool that turns Claude into a hands-on collaborator inside your real codebase.

What Claude Code is

- A command-line tool (`cLaude`) that runs an agentic coding loop in your terminal, operating directly on your project files
- Reads, writes, and edits files, runs shell commands, and uses git - taking real actions rather than just suggesting text
- Powered by Claude models (Opus, Sonnet, Haiku families), it plans multi-step work and iterates until a task is done
- Meets you where you already work: the terminal, your existing repo, your existing toolchain
- Also embeddable in IDEs (VS Code, JetBrains) and scriptable via the Agent SDK, but the CLI is the core experience

Who it is for

- Developers who live in the terminal and want an agent that can act, not just chat
- Engineers handling real tasks: feature work, refactors, bug fixes, test writing, codebase exploration
- Teams that want shared, version-controlled project conventions and reproducible agent behavior
- Anyone automating coding workflows in CI or scripts via the headless / SDK modes
- Comfortable with code review: you stay the reviewer and approver of what the agent does

The terminal-first agentic model

- "Agentic" means Claude decides which tools to call (read, edit, bash, search) and chains them toward a goal
- It explores the codebase on demand instead of needing you to paste files into a chat window
- Each meaningful action (editing files, running commands) is gated by a permission prompt unless pre-approved
- The loop is observable: you see the plan, the tool calls, and the results as they happen
- You can interrupt, redirect, or correct course at any point with Esc and follow-up messages

Install and first run

- Requires Node.js; install the CLI globally, then launch it from inside a project directory:

```
npm install -g @anthropic-ai/claude-code  
cd my-project  
claude
```

- On first run you authenticate (Claude subscription or an Anthropic API key)
- It works best when run at the root of a git repository so it can see structure and use version control
- Run `/init` once to have Claude generate a starter `CLAUDE.md` describing your project
- Use `/help` to list commands and `claude -p "..."` for one-shot headless prompts

CLAUDE.md - project memory

- A markdown file Claude auto-loads as context: conventions, commands, architecture notes, and do/don'ts
- Lives in the repo root (shared, committed) and can be nested in subdirectories for scoped guidance
- A user-level `~/ .claude/CLAUDE.md` applies your personal preferences across all projects
- Keep it concise and high-signal - it is prepended to context, so bloat costs tokens

CLAUDE.md - project memory (cont.)

```
# Project: payments-api
```

```
## Commands
```

- Test: `npm test`
- Lint: `npm run lint` (run before committing)

```
## Conventions
```

- TypeScript strict mode; no `any`
- Prefer small, pure functions; colocate tests as `\*.test.ts`

settings.json and permissions

- `settings.json` configures behavior: permission rules, environment variables, hooks, model, and more
- Scopes: user (`~/.claude/settings.json`), project (`.claude/settings.json`, committed), and local (`.claude/settings.local.json`, gitignored)
- Permission rules allow or deny specific tools and commands so routine actions skip the prompt

```
{
  "permissions": {
    "allow": ["Bash(npm test:*)", "Read", "Edit"],
    "deny": ["Bash(rm -rf:*)", "Read(./env)"]
  }
}
```

settings.json and permissions (cont.)

- Permission modes shape the session: default (prompt), `acceptEdits`, `plan` (read-only planning), and a full-auto bypass mode for sandboxes

Extending Claude Code

- MCP (Model Context Protocol): connect external servers (databases, issue trackers, browsers, internal APIs) as tools and data sources
- Subagents: specialized agent configs with their own prompt and tool scope, used to delegate focused subtasks
- Skills: packaged instructions and assets Claude loads on demand when a task matches their description
- Hooks: shell commands the harness runs automatically on events (e.g., format code after every edit)
- Plugins bundle skills, subagents, hooks, MCP servers, and slash commands for one-step sharing

Extending Claude Code (cont.)

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit", "hooks": [
        { "type": "command", "command": "prettier --write \"$CLAUDE_FILE_PATHS\"" }
      ] }
    ]
  }
}
```

Plan mode and staying in control

- Plan mode lets Claude investigate and draft a step-by-step approach without touching files
- You review and approve the plan before any edits run - good for large or risky changes
- Combine with git: Claude can stage, diff, and commit, and you review changes like any teammate's PR
- Context is managed with `/clear` to reset and `/compact` to summarize long sessions

How it differs from chat assistants and IDE autocomplete

- vs chat assistants (web/app): Claude Code acts on real files and runs commands instead of returning text you copy-paste
- vs IDE autocomplete (e.g., Copilot-style): autocomplete predicts the next lines as you type; Claude Code owns whole multi-file, multi-step tasks
- It has agency over tools and the filesystem, with permissions and git as the safety and review layer
- It carries durable project context via CLAUDE.md and config rather than starting cold each time
- It is scriptable and composable - usable in pipelines and other programs through the Agent SDK

Key takeaways

- Claude Code is an agentic, terminal-first coding tool that takes real actions in your repo, gated by permissions
- CLAUDE.md plus `settings.json` give Claude durable project context and controlled, reproducible behavior
- MCP, skills, subagents, hooks, and plugins extend it; plan mode keeps you in control of big changes
- It differs from chat (it acts, not just suggests) and from autocomplete (whole tasks, not next-line predictions)
- Install with npm, run `claude` in a git repo, `/init` your CLAUDE.md, and review its work like a teammate's

02 · Claude Code Under the Hood

How Anthropic's Claude Code CLI turns a chat model into an autonomous coding agent: the agent loop, its tools, how it reads your repo, the harness around the model, and the model itself.

What Claude Code Actually Is

- A terminal-based coding agent: you type a request, it edits files, runs commands, and reports back.
- It is a *harness* (the CLI program) wrapped around a *model* (a Claude model) plus a *tool set* the model can call.
- The model does not touch your machine directly - every action goes through tools the harness exposes and gates.
- It is agentic: given one prompt it can take many steps on its own until the task is done.
- Also available as the **Agent SDK** (TypeScript/Python) so you can embed the same loop in your own programs.

The Agent Loop

- The core cycle is: read context → plan → call a tool → observe the result → iterate.
- Each turn, the model sees the conversation so far plus tool outputs, then decides the next single action.
- It keeps looping - edit, run tests, read the error, fix, re-run - until it judges the task complete or needs you.
- "Observe" matters: tool results (file contents, command stdout/exit codes) feed back into the next decision.
- The loop ends when the model produces a final text answer with no further tool calls, or hits a permission/stop boundary.

The Tool Set

- **Read** - read a file (or image/PDF/notebook) by absolute path, with line numbers.
- **Edit / Write** - exact string replacement in a file, or create/overwrite a whole file.
- **Bash** - run shell commands; the working directory persists, shell env does not.
- **Grep** - fast content search (ripgrep-backed) by regex across the repo.
- **Glob** - find files by path pattern (e.g. `**/*.ts`).
- **Task / subagents** - spin off a focused sub-agent for a scoped chunk of work.
- Plus web search/fetch, TodoWrite for task tracking, and any tools added via MCP.

How It Reads the Repo

- It does not pre-ingest the whole codebase; it explores on demand using Grep, Glob, and Read.
- This keeps the context window focused – it pulls in only the files relevant to the current task.
- At startup it loads `CLAUDE.md` files (project root, parent dirs, and `~/ .claude/`) as persistent project memory.
- `CLAUDE.md` is where you record conventions, commands, and gotchas so you don't repeat them every session.

How It Reads the Repo (cont.)

```
# CLAUDE.md
## Commands
- Build: `npm run build`
- Test a single file: `npm test -- path/to/file.test.ts`
## Conventions
- Use absolute imports from `@/`
- Never edit files under `generated/`
```

The System Prompt & Harness

- A built-in system prompt tells the model it is Claude Code, how to use tools, and the house style (concise, minimal, no needless preamble).
- The harness injects environment context: working directory, OS, git status, and your `CLAUDE.md` memory.
- It also manages the context window - summarizing/compacting older turns when the conversation grows long.
- **Hooks** let you run your own shell commands at lifecycle points (e.g. before a tool runs, after the model stops).

The System Prompt & Harness (cont.)

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "npm run lint --silent" }]
    }]
  }
}
```

Permissions & Plan Mode

- Every potentially-destructive action (writes, arbitrary Bash) is gated by a permission check before it runs.
- **Permission modes**: default (prompt as needed), accept-edits, plan (read-only - investigate and propose, no changes), and bypass (skip prompts - use with care).
- You pre-approve or deny patterns in `settings.json` so common safe commands stop prompting.
- **Plan mode** is ideal for "look before you leap": the agent researches and writes a plan, then waits for your go-ahead.

Permissions & Plan Mode (cont.)

```
// .claude/settings.json
{
  "permissions": {
    "allow": ["Bash(npm run test:*)", "Read(./src/**)"],
    "deny": ["Bash(rm -rf:*)", "Read(./env)"]
  }
}
```

Extending Claude Code

- **MCP (Model Context Protocol)** - connect external servers (databases, APIs, browsers) that expose extra tools and resources.
- **Skills** - packaged instructions/scripts the agent loads on demand when a task matches their trigger.
- **Subagents** - named, scoped agents with their own prompt and tool subset, dispatched for sub-tasks to keep the main context clean.
- **Plugins** - bundles that ship slash commands, skills, hooks, and MCP servers together for easy install/sharing.
- **Slash commands** - reusable prompt templates (built-in like `/init`, `/review`, or your own project commands).

The Model Behind It

- Claude Code runs on a Claude model (the Opus / Sonnet / Haiku family), chosen for strong tool-use and long-context reasoning.
- The model decides *what* to do; the harness decides *whether it's allowed* and actually executes it.
- Large context windows let it hold many files, tool outputs, and your `CLAUDE.md` at once.
- You can switch models (e.g. a heavier model for hard reasoning, a lighter one for speed/cost) and the harness stays the same.
- The same model+loop pattern powers the Agent SDK, so behavior is consistent between the CLI and embedded use.

Key Takeaways

- Claude Code = harness + model + gated tools running an iterative read→plan→act→observe loop.
- It reads your repo *lazily* (Grep/Glob/Read) and remembers context via `CLAUDE.md`.
- Safety comes from permissions and plan mode - the model proposes, the harness gates, you stay in control.
- Extend it with MCP, skills, subagents, hooks, and plugins instead of changing the core.
- The model reasons and chooses actions; the harness enforces rules and runs them - keep that separation in mind.

03 · CLAUDE.md & Plan Mode

How Claude Code remembers your project across sessions (CLAUDE.md) and how it investigates safely before touching code (Plan Mode).

What CLAUDE.md is

- A plain-markdown file Claude Code loads automatically into context at the start of a session.
- Acts as persistent "project memory" so you don't re-explain conventions every time.
- Lives alongside your code (typically the repo root) and is meant to be committed and shared with the team.
- Treated as instructions/context, not executable config - keep it short and high-signal, since it consumes context budget.
- You can add to it on the fly: prefix a message with `#` and Claude offers to write that note into a CLAUDE.md file.

What to put in CLAUDE.md

- Build / test / lint commands the project actually uses (e.g. `pnpm test`, `make lint`).
- Code style and conventions: formatting, naming, preferred libraries, patterns to follow or avoid.
- Architecture orientation: where key modules live, how the pieces fit together.
- Workflow rules: branching, commit message format, "run tests before committing."
- Gotchas and non-obvious constraints (flaky test, required env vars, things that look wrong but are intentional).

What to put in CLAUDE.md (cont.)

Project: Billing Service

Commands

- Test: ``pnpm test``
- Lint/format: ``pnpm lint && pnpm format``

Conventions

- TypeScript strict mode; no ``any``.
- Use the existing ``Result<T>`` type instead of throwing.

Notes

- ``legacy/`` is frozen - do not refactor.

The memory hierarchy

- Claude Code reads CLAUDE.md from multiple locations and merges them, more specific layered over more general.
- **Enterprise**: a managed system-level policy file, deployed by an admin to all users on a machine.
- **User** (`~/ .claude/CLAUDE.md`): your personal preferences that apply across every project.
- **Project** (`./CLAUDE.md` at repo root): shared, committed team memory for that codebase.
- **Local / nested**: a `CLAUDE.md` in a subdirectory is pulled in when you work in that part of the tree.

How the files combine

- All applicable layers are loaded together; nothing fully "overrides" by replacing - they accumulate as context.
- More specific files (project, then subdirectory) sit closer to the work and naturally take precedence in practice.
- Use **user** memory for habits ("always explain before large edits"), **project** memory for team facts.
- `CLAUDE.local.md` was the old way to keep personal/project notes out of git; the recommended path now is the user-level file plus `.gitignore`.
- Keep the project file lean - anything machine-specific or secret-adjacent belongs in user or untracked files.

Imports and keeping it maintainable

- A CLAUDE.md can pull in other files with `@path/to/file.md`, so you can split memory into focused pieces.
- Good for large repos: a root file that imports per-area docs instead of one giant file.
- Run `/init` in a new project to have Claude scan the codebase and draft a starter CLAUDE.md.
- Use `/memory` to open and edit the active memory files directly.
- Review it periodically - stale instructions actively mislead the model.

See `@docs/architecture.md` and `@docs/testing.md` for details.

What Plan Mode is

- A mode where Claude investigates and proposes an approach but is blocked from editing files or running mutating commands.
- Intended for read-only exploration first: read code, search, reason - then commit to a plan before any change lands.
- Toggle it during a session by cycling permission modes (Shift+Tab); you can also start a session in plan mode.
- It's one of several permission modes, alongside normal (prompt-on-action), accept-edits, and bypass-permissions.
- Reduces the risk of Claude charging ahead and editing the wrong thing before you've agreed on direction.

The Plan Mode workflow

- Claude reads and analyzes the relevant code without modifying anything.
- It presents a concrete plan: what it will change, where, and in what order.
- You review and approve, reject, or ask for revisions - execution only begins after approval.
- On approval, Claude typically exits plan mode and switches to making the edits.
- Best for ambiguous or high-blast-radius tasks (refactors, migrations, anything touching many files).

Where it fits with permissions

- Permission modes govern how much Claude can do without asking; plan mode is the most restrictive useful one.
- `accept-edits` auto-approves file edits for fast iteration on trusted work.
- `bypass-permissions` skips prompts entirely - only for sandboxed/throwaway contexts you trust.
- Fine-grained `allow / deny / ask` rules in `settings.json` complement modes for specific tools and commands.

Where it fits with permissions (cont.)

```
{
  "permissions": {
    "allow": ["Bash(git diff:*)", "Bash(pnpm test:*)"],
    "deny": ["Bash(rm -rf:*)", "Read(../.env)"]
  },
  "defaultMode": "plan"
}
```

Key takeaways

- CLAUDE.md is persistent, committed project memory - put commands, conventions, architecture, and gotchas there, kept short.
- Memory layers (enterprise → user → project → subdirectory) merge, with more specific files effectively winning.
- Use `/init` to bootstrap, `/memory` to edit, `@imports` to keep it modular, and `#` to capture notes mid-session.
- Plan Mode keeps Claude read-only so it proposes a plan you approve before any edit happens.
- It's one point on the permission-mode spectrum (plan → normal → accept-edits → bypass), backed by allow/deny rules in settings.json.

04 · Permissions

How Claude Code decides which tool actions run automatically, which need your approval, and which are blocked outright.

Why permissions exist

- Claude Code can read files, edit code, run shell commands, and call MCP tools - all of which can have real consequences.
- The permission system is the guardrail: it gates *actions* (tool calls), not Claude's reasoning or text.
- Goal is to keep risky operations (deleting files, pushing code, running arbitrary commands) under human control by default.
- Read-only and clearly-safe actions can be pre-approved so you are not interrupted constantly.
- The model proposes a tool call; the permission layer decides allow / ask / deny before it ever executes.

The three rule behaviors

- Every permission rule maps a tool action to one of three behaviors: **allow**, **ask**, or **deny**.
- **allow** - the action runs automatically with no prompt.
- **ask** - Claude pauses and requests your confirmation before running (the interactive default for most state-changing actions).
- **deny** - the action is blocked entirely and cannot be run, even if you would have approved it.
- Precedence: **deny wins over ask, and ask wins over allow** - the most restrictive matching rule applies.
- Deny rules are the right tool for hard guardrails (e.g. never read secrets, never run a destructive command).

Permission modes

- A **mode** sets the overall posture for a session, layered on top of your individual rules.
- **default** - normal behavior: safe actions may be allowed, state-changing actions prompt with "ask".
- **acceptEdits** - auto-accepts file edits (Edit/Write) so you are not prompted per change; other actions still follow their rules.
- **plan** - read-only research mode: Claude can explore and propose a plan but cannot edit files or run side-effecting commands.
- **bypassPermissions** - skips all permission checks and runs everything without prompting; powerful and dangerous, use only in trusted/sandboxed contexts.
- Switch modes interactively (Shift+Tab cycles through them) or set a starting mode via config.

Plan mode in practice

- Plan mode lets Claude read the codebase, search, and analyze without making changes.
- Useful when you want a strategy or review first, then approve execution as a separate step.
- Claude presents a proposed plan and waits; you accept it to move into an editing-capable mode.
- Nothing is written and no side-affecting shell commands run while you are in plan mode.
- Great default for unfamiliar codebases or high-stakes changes where you want to look before you leap.

Where rules live - settings.json

- Permissions are configured in `settings.json` files, merged from several layers.
- Typical layers: enterprise-managed policy, user-level (`~/.claude/settings.json`), project (`.claude/settings.json`), and local project overrides (`.claude/settings.local.json`).
- `settings.local.json` is meant for personal, machine-specific rules and is usually git-ignored; project `settings.json` is shared with the team.
- More specific / more restrictive layers and rules take precedence; enterprise policy can enforce rules users cannot override.
- The `permissions` block holds `allow`, `ask`, and `deny` arrays plus related options.

Where rules live - settings.json (cont.)

```
{
  "permissions": {
    "allow": [
      "Read",
      "Bash(npm run test:*)",
      "Bash(git status)"
    ],
    "ask": [
      "Bash(git push:*)"
    ],
    "deny": [
      "Read(./env)",
      "Bash(rm -rf:*)"
    ]
  }
}
```

Tool-scoped rules and matchers

- A rule names a tool, optionally with a parameter pattern in parentheses: `Tool(specifier)`.
- `Bash(npm run test:*)` matches `npm run test` commands; the `:*` is a prefix wildcard for arguments.
- File tools can scope by path: `Edit(src/**)` or `Read(./secrets/**)` to allow or deny based on location.
- A bare tool name like `Read` or `WebFetch` matches all uses of that tool.
- MCP tools follow the `mcp__<server>__<tool>` naming, so you can allow or deny specific MCP servers/tools.
- Match carefully: overly broad allow rules (e.g. allowing all of `Bash`) defeat the purpose of the guardrails.

Allowlists and reducing prompts

- An allowlist is just your `allow` array - actions you have decided are safe to run unattended.
- Approving an action interactively can offer to remember it, which appends a rule to your settings.
- Good allowlist candidates: read-only inspection (`git status`, `git diff`, `ls`), running tests, type-checks, linters.
- Keep destructive or networked actions (pushes, deploys, `rm`, `curl` to arbitrary hosts) on **ask** or **deny**.
- Curating a tight allowlist is the main way to cut prompt fatigue without lowering your safety posture.

Allowlists and reducing prompts (cont.)

```
{  
  "permissions": {  
    "allow": [  
      "Bash(git diff:*)",  
      "Bash(npm run lint)",  
      "Bash(npm run typecheck)"  
    ]  
  }  
}
```

Safe defaults and the trust prompt

- Out of the box, Claude Code prompts for state-changing actions rather than running them silently.
- The first time you open a new project (a new directory) Claude asks you to confirm you trust that folder before acting.
- File edits, shell commands, and network fetches are gated; pure reads inside the working directory are lighter-touch.
- Deny rules and managed enterprise settings let admins enforce non-negotiable boundaries across a team.
- Treat `bypassPermissions` as an explicit, deliberate choice - best confined to isolated sandboxes or CI, never casual use.

Hooks and the Agent SDK

- **Hooks** run your own shell commands at lifecycle points and can act as a programmatic gate; a `PreToolUse` hook can inspect a pending tool call and approve or block it.
- This lets you enforce policy with custom logic beyond static allow/deny patterns.
- Permissions also apply to **subagents**, which operate under the same rule set and modes as the main session.
- The **Agent SDK** exposes the same permission model programmatically, including a callback (`canUseTool`) to decide on tool calls in code.
- This makes the security model consistent whether you run Claude Code interactively or embed it in an automated pipeline.

Hooks and the Agent SDK (cont.)

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "/usr/local/bin/check-cmd.sh" }
        ]
      }
    ]
  }
}
```

Key takeaways

- Permissions gate *tool actions* via three behaviors - allow, ask, deny - where deny always wins.
- Modes set the session posture: default, acceptEdits, plan (read-only), and the dangerous bypassPermissions.
- Configure rules in layered `settings.json` / `settings.local.json` using tool-scoped patterns like `Bash( ... )` and path matchers.
- Curate a tight allowlist for safe, repetitive actions to reduce prompts without weakening safety.
- For programmatic control, use `PreToolUse` hooks or the Agent SDK's `canUseTool` callback; the model is safe-by-default and asks before changing state.

05 · Effort & Context Windows

How Claude Code spends two scarce budgets - the context window (what the model can "see") and thinking effort (how hard it reasons) - and how to manage both for speed, depth, and cost.

What the context window is

- The context window is the total token budget for a single conversation: everything the model can read at once.
- It holds the system prompt, your CLAUDE.md, tool definitions, your messages, and every tool result (file reads, command output, search hits).
- It is finite and shared - every token spent on one thing is unavailable for another.
- When it fills up, older turns must be summarized or dropped; nothing is "remembered" beyond what currently fits.
- Standard Claude Code sessions run on a large window (around 200K tokens for most models); a 1M-token option exists for select models.

What actually consumes context

- File reads - the biggest variable cost. Reading a 3,000-line file costs far more than reading the 40 lines you needed.
- Tool output - `npm test`, `git log`, build logs, and large command dumps land verbatim in context.
- MCP servers - each connected server injects its tool schemas (and sometimes resources) at startup, before you do anything.
- CLAUDE.md files - loaded automatically from project, parent, and `~/ .claude/`; useful, but always-on overhead.
- The running conversation itself - past tool calls and results accumulate turn after turn.

Keeping the window lean

- Prefer targeted reads: use `offset / limit` or search (Grep/Glob) instead of dumping whole files.
- Disable MCP servers you are not using - their schemas cost tokens whether or not you call them.
- Keep CLAUDE.md tight; it is paid on every single turn, so favor pointers over pasted documentation.
- Use subagents for exploratory or noisy work: a subagent burns its own context and returns only a summary to the main thread.
- Check usage with `/context` to see what is occupying the window before it becomes a problem.

Compaction and summarization

- When the window approaches its limit, Claude Code compacts: it summarizes earlier conversation into a condensed form and continues.
- Auto-compaction triggers near capacity; you can also run `/compact` manually at a clean stopping point.
- Compaction is lossy - fine details, exact file contents, and earlier reasoning can be lost or blurred.
- Best practice: compact deliberately between tasks rather than mid-task, so the summary captures stable conclusions.
- `/clear` is the harder reset - it wipes the conversation entirely; use it when starting unrelated work.

Subagents as a context strategy

- A subagent runs in its own separate context window and reports back a short result.
- This keeps large, throwaway output (searching a big repo, reading many files) out of your main conversation.
- The main thread pays only for the subagent's final summary, not for everything it read along the way.
- Trade-off: subagents cannot see your live conversation, so give them a complete, self-contained task.
- Ideal for parallel investigation, codebase exploration, and any step that generates a lot of noise.

The 1M context option

- Some models offer a 1M-token (1 million) context window - roughly five times the standard size.
- It lets you hold very large codebases, long transcripts, or many files in view at once without compacting.
- It is a separate capability tied to specific model variants (often surfaced with a `[1m]` marker on the model id).
- Larger context typically costs more per token and can be slower; do not reach for it by default.
- Use it when the task genuinely requires breadth - wide refactors, cross-file analysis - not as a substitute for disciplined reads.

Effort and thinking budget

- Separate from context: "effort" (thinking budget) controls how much internal reasoning the model does before answering.
- More thinking improves multi-step reasoning, planning, and tricky debugging - at the cost of latency and tokens.
- In Claude Code, extended thinking is invoked with prompts like "think", "think hard", or "ultrathink" to escalate depth.
- Higher effort consumes context too - reasoning tokens occupy the same window as everything else.
- Match effort to the task: quick edits and lookups need little; architecture decisions and subtle bugs reward more.

Speed vs depth trade-off

- Low effort = fast, cheap, good for mechanical or well-specified changes.
- High effort = slower, pricier, but stronger on ambiguity, planning, and correctness-critical work.
- Plan mode pairs well with higher effort: think first, get a plan, then execute without burning depth on simple steps.
- Over-thinking simple tasks wastes time and tokens; under-thinking hard ones produces confident wrong answers.
- Tune dynamically - escalate when stuck, dial back once the path is clear.

Practical configuration

- A focused CLAUDE.md keeps per-turn overhead low while steering behavior:

```
# Project: payments-api
- Run tests with `pnpm test`, not npm.
- Source lives in `src/`; never edit generated files in `dist/`.
- Prefer reading specific files over `cat`-ing whole directories.
```

- Permission rules in settings.json reduce noisy approval round-trips that clutter a session:

Practical configuration (cont.)

```
{  
  "permissions": {  
    "allow": ["Bash(pnpm test:*)", "Read(src/**)"],  
    "deny": ["Read(.env)"]  
  }  
}
```

- A hook can auto-trim noisy output before it ever reaches context:

Practical configuration (cont.)

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{ "type": "command", "command": "tail -n 100" }]
      }
    ]
  }
}
```

Key takeaways

- Context and effort are two distinct budgets – manage both, not just one.
- Most context waste is avoidable: targeted reads, trimmed output, fewer idle MCP servers, lean CLAUDE.md.
- Compact between tasks and use subagents to keep heavy work out of the main thread.
- Reach for 1M context and high effort intentionally, when breadth or depth truly demands it.
- Right-size effort to the task: fast for the simple, deep for the hard.

06 · Skills

Skills are model-invoked capabilities packaged as folders that Claude Code reads and runs on its own when a task matches, without you typing a command.

What a Skill is

- A reusable capability packaged as a folder containing a `SKILL.md` file plus any supporting assets (scripts, templates, reference docs).
- The `SKILL.md` holds YAML frontmatter (`name`, `description`) and a Markdown body with instructions Claude follows when the skill activates.
- Skills extend what Claude can do for a specialized, repeatable workflow – e.g. filling a PDF form, applying a house style, scaffolding a component.
- They are "progressive disclosure": only the name and description load up front; the full body and assets are read only when the skill is actually used.

Anatomy of a SKILL.md

- The frontmatter `description` is the most important field - it is the only text Claude sees when deciding whether to invoke the skill.
- The body can reference bundled files by relative path; Claude reads them as needed instead of loading everything at once.

```
---  
  
name: pdf-form-filler  
description: Fill and flatten PDF forms from a JSON  
             data file. Use when the user asks to populate a PDF  
             form or generate a filled PDF.  
  
---
```

PDF Form Filler

1. Read the field map in ``reference/fields.md``.
2. Run ``scripts/fill.py <template.pdf> <data.json>``.
3. Return the path to the output PDF.

How Claude decides to use one

- Skills are **model-invoked**: Claude chooses to trigger one based on the conversation, not because you explicitly called it.
- At startup, Claude is given each skill's `name` and `description` as lightweight metadata.
- When your request semantically matches a description, Claude opens that `SKILL.md` and follows its instructions.
- A precise, trigger-oriented description ("Use when the user asks to...") makes activation reliable; a vague one means the skill may never fire.
- Multiple skills can be relevant; Claude picks based on the best match and may chain its steps with normal tool use.

Where Skills live

- **Personal skills:** `~/ .claude/skills/<skill-name>/SKILL.md` - available across all your projects.
- **Project skills:** `.claude/skills/<skill-name>/SKILL.md` - committed to the repo and shared with the team.
- **Plugin skills:** bundled inside an installed plugin and namespaced as `plugin-name:skill-name`.
- Each skill is its own subfolder; the folder name typically matches the skill `name`.
- Assets sit alongside `SKILL.md` in that folder and are referenced by relative path.

Skills vs slash commands

- **Slash commands** are *user-invoked*: you type `/name` to run a prompt template on demand.
- **Skills** are *model-invoked*: Claude decides to use them when the task fits, with no typing required.
- A slash command is essentially a single Markdown prompt file; a skill is a folder that can carry scripts, data, and multiple reference files.
- Use a slash command for an action you want to trigger deliberately; use a skill for a capability Claude should reach for automatically.
- They are complementary - a slash command can be a deliberate entry point while skills handle background know-how.

Skills vs other extension points

- **CLAUDE.md** is always-on project context and conventions; a skill loads only when relevant, keeping the base context lean.
- **MCP servers** add external *tools and data sources* over a protocol; skills add *instructions and workflows* using the tools Claude already has.
- **Subagents** are separate context windows for delegated work; a skill is guidance executed within the current agent (though a skill can drive multi-step tool use).
- **Hooks** are deterministic shell commands the harness runs on events; skills are model-driven and only fire when Claude judges them relevant.

Authoring good Skills

- Write the `description` for the decision moment: state what the skill does and the situations that should trigger it.
- Keep the `SKILL.md` body focused; push long details into separate reference files the body links to, preserving progressive disclosure.
- Bundle executable helpers (Python/shell scripts) for deterministic steps instead of asking Claude to redo them by hand each time.
- Test by describing a matching task in plain language and confirming Claude reaches for the skill on its own.
- Commit project skills to version control so the whole team gets the same behavior.

Distribution via plugins

- Skills can ship inside **plugins**, installed from a marketplace, so capabilities are shareable beyond a single repo.
- Plugin skills appear namespaced (`plugin:skill`) to avoid collisions with personal or project skills.
- A single plugin commonly bundles related skills, slash commands, hooks, and MCP servers together.
- This makes skills a portable unit of expertise you can publish, version, and reuse across teams.

Key takeaways

- A skill is a folder (`SKILL.md` + assets) that packages a reusable, specialized workflow.
- Skills are model-invoked - Claude triggers them automatically - while slash commands are user-invoked by typing `/name`.
- Only `name` and `description` load up front; the body and files load on demand (progressive disclosure), so a sharp description is critical.
- They live in `~/.claude/skills/` (personal), `.claude/skills/` (project), or inside plugins (namespaced).
- Skills complement `CLAUDE.md`, MCP, subagents, and hooks rather than replacing them: they add on-demand instructions and assets.

07 · Hooks

Hooks let you run your own shell commands automatically at defined points in Claude Code's lifecycle, so you can enforce policy, automate chores, and shape the agent's behavior deterministically.

What hooks are

- User-defined shell commands that Claude Code runs automatically at specific lifecycle events
- They are deterministic: the harness runs them every time, unlike asking the model to "remember" to do something
- Configured in `settings.json` (not in `CLAUDE.md`, which is just guidance for the model)
- Each hook receives a JSON payload on stdin describing the event and returns signals via exit code or JSON on stdout
- Good for things you want guaranteed: formatting, linting, audit logs, guardrails

The lifecycle events

- `PreToolUse` - fires before a tool runs; can inspect and block the call
- `PostToolUse` - fires after a tool succeeds; good for reacting to changes (e.g. format an edited file)
- `UserPromptSubmit` - fires when you submit a prompt, before the model sees it
- `SessionStart` / `SessionEnd` - fires when a session begins or ends
- `Stop` / `SubagentStop` - fires when the main agent (or a subagent) finishes responding
- `Notification` and `PreCompact` - fire on notifications and before context compaction

How hooks are configured

- Live under a top-level `hooks` key in `settings.json` (project, local, or user scope)
- Each event maps to an array of matcher groups; each group has a `matcher` and a list of `hooks`
- The `matcher` selects which tools trigger the hook (e.g. `Edit`, `Write`, or a regex like `Edit|Write`)
- Events without tools (like `SessionStart`) ignore the matcher
- Each hook entry has `type: "command"` and a `command` string

How hooks are configured (cont.)

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "prettier --write \"$CLAUDE_FILE_PATHS\"" }
        ]
      }
    ]
  }
}
```

The input payload

- Hooks receive a JSON object on stdin every time they fire
- Common fields: `session_id`, `hook_event_name`, `cwd`, and the transcript path
- Tool events include `tool_name` and `tool_input` (and `tool_response` for `PostToolUse`)
- Read it with `jq` or any language to make decisions based on what the tool is doing
- Example: inspect the command a `Bash` call is about to run before allowing it

```
#!/usr/bin/env bash
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command')
echo "$cmd" | grep -qE '\brm -rf\b' && { echo "Blocked: rm -rf" >&2; exit 2; }
```

Blocking vs non-blocking

- Exit code `0` - success; the action proceeds (stdout is informational)
- Exit code `2` - blocking error; the action is stopped and stderr is fed back to Claude
- Other non-zero codes - non-blocking error; surfaced to the user but the flow continues
- Only certain events can actually block: `PreToolUse` blocks the tool call, `UserPromptSubmit` blocks the prompt, `Stop` can force the agent to keep working
- `PostToolUse` runs after the fact, so it cannot undo the tool, but a non-zero exit still feeds feedback to Claude

JSON output for finer control

- Instead of exit codes, a hook can print a JSON object to stdout for richer control
- `continue`, `stopReason`, and `suppressOutput` control overall flow
- For `PreToolUse`, a `permissionDecision` of `allow`, `deny`, or `ask` overrides the normal permission prompt
- `UserPromptSubmit` and `SessionStart` can inject `additionalContext` into the conversation
- This lets a hook approve trusted commands automatically or add reason text when denying

JSON output for finer control (cont.)

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Edits to .env are not allowed"
  }
}
```

Use case - format and lint on save

- Attach a `PostToolUse` hook on `Edit|Write` to run a formatter on changed files
- Use `$CLAUDE_FILE_PATHS` (or parse `tool_input` from stdin) to target only what changed
- Pair with a linter that exits non-zero so Claude sees and fixes the errors
- Keeps the codebase consistent without the model having to remember to format
- Faster and more reliable than prompting "always run prettier afterward"

Use case - guardrails

- A `PreToolUse` hook on `Bash` can scan and block dangerous commands before they run
- A `PreToolUse` hook on `Edit|Write` can protect sensitive paths (secrets, lockfiles, CI config)
- Return exit `2` or a `deny` decision with a clear reason so Claude understands and adapts
- `UserPromptSubmit` hooks can reject prompts that violate policy or inject required context
- These run in the harness, so the model cannot talk its way past them

Scope, ordering, and safety

- Hooks resolve from user (`~/.claude/settings.json`), project (`.claude/settings.json`), and local settings
- Multiple matching hooks for an event all run; non-blocking ones run in parallel
- Hooks execute with your full user permissions - treat them like any script you trust
- Newly edited hook config is not applied mid-session for safety; review changes before they take effect
- Test commands standalone first, and quote variables to handle paths with spaces

Key takeaways

- Hooks make selected behaviors deterministic instead of relying on the model's good intentions
- Configure them per event in `settings.json` with matchers that select tools
- Exit code `2` (or a JSON `deny`) blocks; `PreToolUse` and `UserPromptSubmit` are your real gates
- Classic wins: auto-format on `PostToolUse`, command/path guardrails on `PreToolUse`
- They run as you, with your permissions - keep them simple, safe, and tested

08 · MCP Overview

How the Model Context Protocol lets Claude Code reach beyond its built-in tools to talk to external systems, data sources, and services through standardized servers.

What MCP is

- MCP (Model Context Protocol) is an open standard for connecting LLM applications to external tools and data through a uniform interface.
- In Claude Code, MCP is the plugin layer for capabilities that are not built in - databases, issue trackers, browsers, internal APIs, SaaS products.
- A "host" (Claude Code) launches or connects to one or more "MCP servers," each exposing a set of capabilities.
- The protocol is transport-agnostic and JSON-RPC based, so the same server can serve many different MCP clients, not just Claude Code.
- The point: you write or install a server once, and Claude can use it without custom glue code in the model itself.

The three capability types

- **Tools** - callable functions the model can invoke (e.g. `query_database`, `create_ticket`). These are the most common and are surfaced to Claude as available actions.
- **Resources** - readable data the server exposes by URI (e.g. a file, a row, a document). Claude Code can list and read these via dedicated resource tools.
- **Prompts** - server-defined prompt templates that surface in Claude Code as slash commands, letting a server ship reusable workflows.
- Tools are model-invoked; resources are typically referenced or read on demand; prompts are user-invoked.
- A single server can expose any mix of the three.

Transports - stdio vs HTTP

- **stdio**: Claude Code spawns the server as a local subprocess and talks over stdin/stdout. Best for local tools, scripts, and anything that runs on your machine.
- **HTTP** (streamable HTTP, the successor to the older SSE transport): Claude Code connects to a server over a URL. Best for remote/hosted servers and shared team services.
- stdio servers start and stop with your Claude Code session; HTTP servers run independently and are reached over the network.
- Remote HTTP servers commonly require auth (headers or OAuth); stdio servers usually inherit your local environment and credentials.
- Choose stdio for "runs here," HTTP for "lives somewhere else."

Adding servers with the CLI

- The fastest way to register a server is `claude mcp add`, which writes the config for you.
- Example (stdio):

```
claude mcp add my-db --scope project \  
  -- npx -y @some/db-mcp-server --readonly
```

- Example (HTTP):

Adding servers with the CLI (cont.)

```
claude mcp add --transport http linear https://mcp.example.com/sse \
  --header "Authorization: Bearer ${TOKEN}"
```

- Useful companions: `claude mcp list` to see configured servers and `claude mcp get <name>` for details.
- The `--` separates Claude's flags from the command/args passed to the server process.

Config scopes & the JSON shape

- Servers can be configured at three scopes: **local** (just you, this project), **project** (shared via committed `.mcp.json`), and **user** (you, across all projects).
- The `.mcp.json` file is the team-shareable form - commit it so everyone gets the same servers.
- A typical entry looks like:

Config scopes & the JSON shape (cont.)

```
{
  "mcpServers": {
    "my-db": {
      "command": "npx",
      "args": ["-y", "@some/db-mcp-server", "--readonly"],
      "env": { "DB_URL": "${DB_URL}" }
    }
  }
}
```

- HTTP servers use `"type": "http"`, a `"url"`, and an optional `"headers"` block instead of `command` / `args`.

Config scopes & the JSON shape (cont.)

- Use `${VAR}` expansion to keep secrets out of committed files.

How MCP tools appear to Claude

- MCP tool names are namespaced as `mcp__<server>__<tool>` so they never collide with built-in tools or other servers.
- Inside Claude Code you can reference a server's prompts as slash commands like `/mcp__<server>__<prompt>`.
- The `/mcp` command shows connected servers, their status, and lets you authenticate servers that need OAuth.
- Because tool names are predictable, you can scope permissions to a whole server (`mcp__linear__*`) or a single tool.
- Servers that fail to start show up as errors in `/mcp` rather than silently disappearing.

Permissions & allowlisting

- MCP tools obey the same permission system as everything else - by default Claude asks before calling them.
- You can pre-approve trusted tools in `settings.json` to cut down on prompts:

```
{
  "permissions": {
    "allow": ["mcp__my-db__query_database"],
    "deny":  ["mcp__my-db__drop_table"]
  }
}
```

- Allow rules can target an entire server (`mcp__my-db__*`) or individual tools; deny rules win over allow.

Permissions & allowlisting (cont.)

- Plan mode and permission modes still apply: nothing executes until it clears your configured permission gate.
- Prefer allowlisting read-only tools and keeping anything destructive behind explicit approval.

Security considerations

- An MCP server is third-party code running with your credentials and environment - treat installing one like installing a dependency.
- Prefer servers you trust or can audit; pin versions and avoid pulling unpinned remote code where possible.
- Watch for prompt-injection: tool results and resources are untrusted input and can try to steer Claude - be cautious about auto-approving tools that act on that content.
- Keep secrets in environment variables referenced from config, never hard-coded in committed `.mcp.json`.
- For remote HTTP servers, verify the endpoint and use proper auth; for stdio servers, remember they inherit your local shell access.
- Use `deny` rules and least-privilege allowlists so a compromised or buggy server can't take destructive actions unprompted.

When to use MCP vs other extension points

- **MCP** - connect to an external system or shared service with a real protocol (DB, API, browser, SaaS).
- **Skills** - package reusable instructions/workflows for Claude; no external process needed.
- **Subagents** - delegate a scoped task to a separate context window.
- **Hooks** - run your own shell commands deterministically on lifecycle events (e.g. format on edit).
- **Plugins** - bundle and distribute any of the above (including MCP servers) as one installable unit.
- Reach for MCP specifically when the capability lives outside Claude Code and benefits from a standardized client/server boundary.

Key takeaways

- MCP is Claude Code's standard way to reach external tools and data via servers that expose tools, resources, and prompts.
- Pick the transport by location: stdio for local subprocesses, HTTP for remote/hosted services.
- Configure with `claude mcp add` or a committed `.mcp.json`, using scopes and `${VAR}` to share safely.
- MCP tools are namespaced (`mcp__server__tool`) and fully governed by the permission system - allowlist read-only, deny destructive.
- Servers run with your access, so treat them as trusted dependencies and stay alert to prompt injection from tool output.

09 · Subagents

How Claude Code delegates work to fresh, isolated agent instances so the main conversation stays focused and parallelizable.

What a subagent is

- A separate Claude instance spawned by the main agent to handle a scoped task
- Runs with its own context window - it does not see your full conversation history, only the prompt it is handed
- Reports back a single result; intermediate steps stay out of the main thread
- Launched via the Task tool; you can also define reusable named agent types
- Think of it as "fork a worker, get an answer, discard the scratch work"

Why delegate - isolated context

- Each subagent gets a clean context window, so noisy exploration doesn't bloat the main conversation
- The main agent only receives the distilled result, keeping its own context lean and on-task
- Useful for searches that read many files: the subagent burns tokens reading, you get the summary
- Reduces the chance the main thread drifts or loses the original goal under a pile of tool output
- Failures or dead ends are contained - they don't pollute the primary conversation

Why delegate - parallelism

- Multiple subagents can run concurrently when their tasks are independent
- Good for fan-out work: search three areas of a codebase at once, then merge findings
- Launch them in a single batch so they execute in parallel rather than one after another
- Wall-clock time drops because reads/searches happen simultaneously
- Only parallelize truly independent work - shared edits must be sequenced to avoid conflicts

Custom agent types

- Define reusable agents as Markdown files in `.claude/agents/` (project) or `~/.claude/agents/` (user)
- Each file has YAML frontmatter plus a system prompt describing the agent's role
- You can restrict an agent's tools and give it a clear "when to use me" description
- The main agent can then invoke that named type instead of writing an ad-hoc prompt each time
- Plugins can also ship agent definitions for the whole team

```
name: test-runner
description: Runs the test suite and summarizes only failures. Use after code changes.
tools: Bash, Read, Grep
```

How invocation works

- The main agent calls the Task tool with a subagent type and a self-contained prompt
- The prompt must include everything the worker needs - it has no access to prior turns
- The subagent's final text response is the only thing returned to the caller
- Subagents cannot spawn deeper subagents in a chain; keep delegation one level
- Be explicit about the expected output shape (e.g. "return absolute file paths and line numbers")

When to delegate

- Open-ended search or research across many files where you only need the conclusion
- Independent subtasks that can run in parallel to save time
- Heavy, token-expensive exploration you want to keep out of the main context
- Repeatable specialized jobs (review, test, lint triage) backed by a custom agent type
- Verification passes you want done with a fresh, unbiased context

When to do it inline

- Quick, single-file edits or a couple of obvious commands - delegation overhead isn't worth it
- Work that depends tightly on the current conversation's nuance and state
- Tasks needing back-and-forth with you; subagents run autonomously to completion
- Sequential edits to shared files, where parallel workers would collide
- When you need to see and steer each step rather than receive only a final summary

Subagents vs. related features

- Skills: packaged instructions/scripts loaded on demand; a subagent can use skills, but they aren't the same thing
- MCP: external tool/data servers; subagents may have MCP tools available within their tool set
- Hooks: shell commands the harness fires on events (e.g. `SubagentStop`) - automation around agents, not agents themselves
- Plan mode: a read-only planning state of one agent; distinct from spawning workers
- Agent SDK: the programmatic way to build your own agents/subagent orchestration outside the CLI

Practical tips

- Write the delegation prompt as a standalone brief, including success criteria and output format
- Scope tools tightly in custom agents - fewer tools means safer, more focused behavior
- Ask for absolute paths and concrete references so results are actionable
- Don't over-delegate trivial work; the round-trip costs latency and tokens
- Use `SubagentStop` hooks if you want automatic checks after a worker finishes

Practical tips (cont.)

```
{
  "hooks": {
    "SubagentStop": [
      { "hooks": [{ "type": "command", "command": "npm run lint --silent" }] }
    ]
  }
}
```

Key takeaways

- Subagents = isolated, disposable workers that return a result and keep the main context clean
- Delegate for broad search, parallel fan-out, and repeatable specialized jobs
- Define custom agent types in `.claude/agents/` with frontmatter, a focused prompt, and limited tools
- Always hand a subagent a self-contained brief - it cannot see your conversation
- Do trivial, stateful, or interactive work inline; delegation shines on independent, token-heavy tasks

10 · Agent Teams

How to coordinate multiple Claude Code agents - subagents and orchestrators - so that work gets split up, run in parallel, and verified instead of being done by one agent in a single context.

What is an agent team?

- An "agent team" is several Claude instances working toward one goal, rather than a single long-running session.
- The core building block is the **subagent**: a separate agent the main session spawns via the Task tool to handle a focused piece of work.
- Each subagent runs in its **own context window** and returns only a final text summary to the caller - its intermediate tokens never pollute the parent's context.
- The main session acts as the **orchestrator**: it plans the work, dispatches subagents, and stitches their results together.
- Teams help with context budget (offload big reads), focus (one job per agent), and parallelism (many agents at once).

Defining reusable subagents

- Custom subagents live as markdown files in `.claude/agents/` (project) or `~/.claude/agents/` (user), with YAML frontmatter.
- The `description` field tells the orchestrator *when* to delegate to that agent; a strong description drives automatic routing.
- You can scope each subagent's tools and give it its own system prompt, so a reviewer agent can be read-only while a builder can edit.

```
name: test-runner
description: Runs the test suite and reports failures. Use after code changes.
tools: Bash, Read, Grep
model: sonnet
```

You run the project's tests, then summarize only the failing cases and the likely cause. Do not attempt fixes.

Orchestration pattern - Fan-out

- **Fan-out** means the orchestrator splits a task into independent chunks and dispatches one subagent per chunk.
- Best when subtasks don't depend on each other: "audit each of these 5 modules," "summarize each log file."
- Launch them in parallel by issuing multiple Task calls **in a single turn** – they then run concurrently.
- Each agent returns a compact result; the orchestrator merges them into one answer.
- Keep each subagent's brief self-contained, since they can't see each other's work.

```
orchestrator [ subagent: audit auth/  
              subagent: audit api/  
              subagent: audit db/    → merge findings
```

Orchestration pattern - Pipeline

- A **pipeline** chains agents so each one's output feeds the next stage.
- Useful when steps are dependent: research → plan → implement → test.
- The orchestrator runs these **sequentially**, passing forward only the distilled result it needs, not raw context.
- This keeps each stage's context clean and lets you swap a specialized agent into any stage.
- Trade-off: no parallelism, so reserve it for genuinely ordered work.

Orchestration pattern - Verify

- The **verify** pattern pairs a "doer" with a separate "checker" agent.
- A second agent with fresh context reviews the first one's output - catching errors the author is blind to.
- Common pairs: implement + code-review, write + fact-check, fix + run-tests.
- Give the verifier read-only tools so it can inspect but not silently "fix" and mask problems.
- The `/security-review` and `/review` slash commands are built-in examples of verifier-style agents.

Running work in parallel

- Parallelism comes from issuing several Task calls in one assistant turn; the orchestrator waits for all to finish.
- Good parallel candidates: independent file reads, multi-target searches, fan-out audits, generating alternatives.
- Watch for **write conflicts** - parallel agents editing the same files collide; keep parallel work read-only or partition the files.
- For heavier isolation, run separate Claude Code sessions in **git worktrees**, each on its own branch.
- Parallelism trades token/cost for wall-clock speed, so use it where the subtasks are truly independent.

Combining subagents into a workflow

- Real workflows mix the patterns: fan-out to gather, pipeline to process, verify to gate the result.
- Encode a repeatable workflow as a **slash command** (`.claude/commands/`) that lays out the steps and which agents to call.
- **Hooks** can automate hand-offs - e.g. a `SubagentStop` or `PostToolUse` hook that kicks off the next step.

Combining subagents into a workflow (cont.)

```
{
  "hooks": {
    "Stop": [
      { "matcher": "", "hooks": [
        { "type": "command", "command": "npm test" }
      ]}
    ]
  }
}
```

- Use **plan mode** first to design the whole team and approve it before any agent starts editing.

Combining subagents into a workflow (cont.)

- Permission rules in `settings.json` set the guardrails the whole team inherits.

Orchestration with the Agent SDK

- For programmatic teams, the **Claude Agent SDK** (TypeScript/Python) lets you build orchestrators in code instead of the CLI.
- You can define subagents, tools, and permission policy, then drive multi-agent loops from a script or service.
- This is the path for CI jobs, batch processing, or wiring a fan-out/verify pattern into an automated pipeline.
- The SDK shares the same primitives as the CLI - system prompts, tools, MCP servers, permission modes - so designs transfer over.
- Use it when you need the team to run unattended; use the CLI when a human stays in the loop.

Practical guidance

- Prefer one clear job per subagent; vague briefs produce vague results since the agent can't ask the parent for clarification mid-run.
- Pass down only what the agent needs, and expect back only a tight summary - that is the whole point of separate contexts.
- Subagents cannot spawn their own subagents, so keep orchestration at the top level.
- More agents means more tokens and cost; reach for a team when context, focus, or parallelism actually pays off - not by default.
- Name and store agents/commands in the repo so the team is version-controlled and shared.

Key takeaways

- A team = an orchestrator plus focused subagents, each in its own clean context returning only a summary.
- Three core patterns: **fan-out** (parallel independent work), **pipeline** (ordered stages), **verify** (separate checker).
- Get parallelism by launching multiple Task calls in one turn; isolate heavy parallel edits with git worktrees.
- Combine patterns into repeatable workflows using slash commands, hooks, plan mode, and `settings.json` guardrails.
- Use the Agent SDK for unattended/programmatic teams; spend extra agents only when they clearly earn their cost.

11 · Skill Creator

How to author your own Claude Code Skills - folder layout, an effective SKILL.md, bundled scripts, testing, and the conventions that make a Skill discoverable and reliable.

What a Skill is

- A Skill is a reusable, model-invoked capability packaged as a folder containing a `SKILL.md` file plus optional supporting files.
- Claude Code reads each Skill's name and description at startup, then decides on its own when a Skill is relevant to the current task and loads its full instructions on demand.
- This is progressive disclosure: only the lightweight metadata sits in context until a Skill is actually triggered, keeping the context window lean.
- Skills differ from `CLAUDE.md` (always-on project memory) and from slash commands (explicit user invocation) - Skills are pulled in by the model when they match.
- They can be personal (`~/ .claude/skills/`), project-scoped (`.claude/skills/`), or shipped inside a plugin.

Skill folder structure

- Each Skill lives in its own directory; the directory name is the Skill's identifier (use lowercase-with-hyphens).
- The only required file is `SKILL.md` at the folder root.
- Supporting files sit alongside it - reference docs, scripts, templates, and example data.
- A common convention is a `references/` subfolder for extra markdown and a `scripts/` subfolder for executables.

Skill folder structure (cont.)

```
.claude/skills/  
├── pdf-extractor/  
│   ├── SKILL.md           # required: metadata + instructions  
│   ├── references/  
│   │   └── advanced.md    # loaded only when needed  
│   ├── scripts/  
│   │   └── extract.py  
│   └── templates/  
│       └── report.md
```

Anatomy of SKILL.md

- A SKILL.md has two parts: YAML frontmatter at the top and Markdown instructions in the body.
- Frontmatter requires `name` and `description`; the body holds the actual guidance Claude follows once the Skill loads.
- `name` should match the folder and be short and descriptive; `description` is the single most important field for discovery.
- The body can be as long as needed - but offload deep detail to `references/` files the body points to, so it loads only when required.

```
---
```

```
name: pdf-extractor
description: Extract text and tables from PDF files. Use when the
  user needs to read, parse, or pull data out of a PDF document.
```

Writing an effective description

- The description is what Claude matches against the task - write it to answer "what does this do AND when should it be used."
- Lead with the capability, then add explicit trigger phrases ("Use when...") that mirror how a user would actually phrase the request.
- Be specific: vague descriptions like "helps with files" rarely fire; concrete nouns and verbs win.
- Mention key terms a user might say (formats, tools, domains) so the match is robust to phrasing.
- Keep it to a couple of sentences - it lives in context for every session, so it must earn its space.

Writing the instruction body

- Write for Claude as the reader: imperative steps, clear ordering, and unambiguous decision points.
- Prefer numbered procedures and short sections over long prose; call out edge cases explicitly.
- Point to bundled scripts and reference files by relative path rather than inlining everything.
- State the expected output format so results are consistent across runs.
- Avoid duplicating what the model already knows well - focus on the project- or domain-specific knowledge it lacks.

Bundling scripts and resources

- Scripts let a Skill do deterministic work (parsing, API calls, transforms) instead of relying on the model to improvise.
- Reference them by relative path from the Skill folder; make executables runnable and document how to call them in the body.
- Keep heavy reference material in separate files and link to them - the model reads them only when the step requires it.
- Declare any runtime prerequisites (Python version, packages, CLI tools) in the instructions so the model can check or install them.
- Remember the model still needs permission to run commands - script execution goes through normal Claude Code permission prompts and allowlists.

Testing your Skill

- Restart or reload Claude Code so it picks up the new Skill folder, then confirm it appears in the available-skills list.
- Test discovery: phrase a request the way a real user would and verify the Skill actually fires without you naming it.
- Test execution: walk through the full procedure, checking that scripts run and reference files load as intended.
- Iterate on the `description` first when a Skill fails to trigger - that is the most common cause of a Skill being ignored.
- Try negative cases too: make sure the Skill does NOT fire on unrelated requests.

Best practices

- One Skill, one job - narrow, focused Skills are easier to match and maintain than sprawling ones.
- Make the description trigger-rich and the body lean; push detail into `references/` to protect context.
- Use scripts for anything that must be exact or repeatable; let the model handle judgment.
- Version and review Skills like code; project Skills in `.claude/skills/` can be committed so the whole team shares them.
- Package related Skills into a plugin when you want to distribute them across repos or teammates.
- Keep names, folders, and the `name` field consistent to avoid confusion.

Key takeaways

- A Skill is a folder with a required `SKILL.md` (YAML `name` + `description`, then Markdown instructions) that Claude loads on demand.
- The `description` drives discovery - make it specific and include "use when" trigger phrases.
- Keep the body lean and offload depth to `references/` and `scripts/` so context stays small.
- Bundle scripts for deterministic work; they still run under normal permission controls.
- Test both that the Skill fires when it should and stays quiet when it shouldn't, tuning the description first.

12 · Plugins Overview

Plugins are installable bundles that package skills, slash commands, hooks, subagents, and MCP servers together so a whole team can extend Claude Code the same way with one command.

What a plugin is

- A plugin is a single distributable unit that bundles one or more Claude Code extensions into a versioned package
- It can contain any mix of: skills, slash commands, hooks, subagents (custom agents), and MCP server definitions
- The point is packaging and sharing - instead of hand-copying files into `.claude/`, you install one plugin and get everything
- Plugins are loaded by Claude Code at startup; their contents appear alongside your own project and user-level config
- Each plugin has a manifest (`plugin.json`) declaring its name, version, and what it provides

The five things a plugin can bundle

- **Skills** - model-invoked capability folders (a `SKILL.md` plus optional scripts/references) that Claude loads on demand
- **Slash commands** - `/name` prompt templates the user triggers explicitly
- **Hooks** - shell commands wired to lifecycle events (e.g. `PreToolUse`, `PostToolUse`, `SessionStart`, `Stop`)
- **Subagents** - specialized agent definitions with their own prompt, tools, and model that Claude can delegate tasks to
- **MCP servers** - connections to external tools/data via the Model Context Protocol, so the plugin can ship integrations (GitHub, a database, an internal API)
- A plugin does not have to include all five - many ship just a skill or just a set of commands

Marketplaces

- A marketplace is a catalog of plugins, defined by a `marketplace.json` that lists available plugins and where to fetch them
- Marketplaces are typically backed by a git repository, so a team can host its own private internal marketplace
- You add a marketplace once, then browse and install any plugin it offers
- Anthropic and the community publish marketplaces; organizations commonly run their own for internal-only plugins
- The marketplace is the discovery layer; the plugin is the actual installed payload

```
# add a marketplace (e.g. a team's internal repo)
/plugin marketplace add your-org/claude-plugins
```

Installing and managing plugins

- The `/plugin` command is the entry point for browsing, installing, enabling, disabling, and removing plugins
- `Install` pulls the plugin from its marketplace; you can then enable or disable it without fully uninstalling
- `Disabling` is useful to silence a noisy plugin or resolve a command-name conflict without losing the install
- Plugins are versioned, so you can update to pick up new commands, skills, or fixes

```
/plugin                # open the interactive plugin menu  
/plugin marketplace add <repo>  
/plugin install <name>@<marketplace>
```

Anatomy of a plugin

- A plugin is a directory with a `plugin.json` manifest at its root describing metadata and contents
- Conventional subfolders hold each extension type, for example:

```
my-plugin/  
  plugin.json  
  commands/      # slash commands (.md)  
  skills/        # skill folders, each with SKILL.md  
  agents/        # subagent definitions  
  hooks/hooks.json # hook event wiring  
  .mcp.json      # MCP server definitions
```

Anatomy of a plugin (cont.)

- Claude Code reads the manifest, then exposes whatever folders exist – missing folders are simply skipped
- Paths inside the plugin can reference `${CLAUDE_PLUGIN_ROOT}` so scripts resolve correctly regardless of install location

How plugins layer with the rest of config

- Claude Code merges configuration from several places: user-level (`~/ .claude/`), project-level (`.claude/`), and installed plugins
- Plugin skills and commands live alongside your own – a project skill and a plugin skill can coexist
- Settings precedence still applies: enterprise managed settings, then user, then project `settings.json` / `settings.local.json`
- Plugin hooks run as part of the same lifecycle events as any hooks you define yourself in `settings.json`
- MCP servers from a plugin behave like MCP servers you'd add manually – they just arrive pre-packaged

Plugins for a team

- Plugins turn "here's how we use Claude Code" into something installable and version-controlled instead of tribal knowledge
- A shared internal marketplace lets every engineer install the same review commands, deploy workflows, and house-style skills
- Bundling hooks lets a team enforce conventions automatically - e.g. run a formatter after edits or block writes to protected paths
- Onboarding shrinks to a couple of `/plugin` commands rather than copying snippets into each person's config
- Updating one plugin in the marketplace rolls improvements out to the whole team

Plugins for a team (cont.)

```
// hooks.json - auto-format edited files team-wide
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "prettier --write \"$CLAUDE_FILE_PATHS\"" }
        ]
      }
    ]
  }
}
```

When to build a plugin (vs. plain config)

- Use a plain `.claude/` skill, command, or `CLAUDE.md` when it's specific to one repo and you don't need to share it
- Reach for a plugin when the same extensions should travel across many repos or many teammates
- Plugins are also right when you want versioning, a changelog, and a clean install/uninstall story
- If you're shipping an MCP integration plus the commands that drive it, bundling them in one plugin keeps them in sync
- Keep plugins focused - a tight bundle around one workflow is easier to adopt than a kitchen-sink plugin

Key takeaways

- A plugin bundles skills, commands, hooks, subagents, and MCP servers into one installable, versioned package
- Marketplaces (often git-backed, including private internal ones) are how plugins are discovered and distributed
- Manage everything through `/plugin` - add a marketplace, install, enable/disable, update
- Plugin contents layer cleanly with your user- and project-level config and follow the same settings precedence
- For teams, plugins standardize workflows and enforce conventions, turning setup into a one-command onboarding step

13 · Cowork

How to treat Claude Code as a collaborator you work alongside - pairing in real time, handing off async work, and keeping a human in the loop at the right checkpoints.

What "Cowork" means here

- Cowork is a working style, not a single button: you and the agent share the same repo, terminal, and task context.
- Claude Code runs as an interactive loop - you steer, it acts, you review, you steer again.
- The unit of collaboration is the session: a conversation plus the file edits, commands, and tool calls it produces.
- Good cowork is about controlling *when* the agent acts autonomously versus *when* it pauses for you.

Plan mode - agree before acting

- Plan mode lets Claude research and propose an approach without editing files or running mutating commands.
- Enter it with Shift+Tab (cycling permission modes) or start a session in plan mode.
- The agent reads code, asks questions, and presents a plan; you approve, edit, or reject before any change lands.
- Ideal first step for cowork: you and the agent align on intent, then switch to an execution mode.
- Reduces wasted work - no large diffs appear until you've signed off on the direction.

Permission modes - the autonomy dial

- Default mode: Claude asks before each consequential action (writes, shell commands, network).
- `acceptEdits`: file edits apply automatically, but commands still prompt - useful for fast iteration on code.
- `plan`: read-only research and proposal, no changes.
- "Bypass permissions" / YOLO-style mode runs without prompts - only for trusted, sandboxed work.
- Cowork is mostly about picking the right point on this dial for the task and your trust level.

Persisting shared context with CLAUDE.md

- `CLAUDE.md` is project memory loaded into every session - your shared rulebook with the agent.
- Put conventions, commands, and gotchas there so each session starts aligned instead of re-learning.
- Scopes stack: enterprise, user (`~/ .claude/CLAUDE.md`), and project files combine.

Persisting shared context with CLAUDE.md (cont.)

```
# CLAUDE.md
## Workflow
- Run `npm test` before proposing a commit.
- Never push to `main`; open a PR instead.
- Ask before editing files under `infra/`.

## Conventions
- TypeScript, 2-space indent, no default exports.
```

- Treat it like onboarding docs for a teammate who forgets between sessions.

Human-in-the-loop checkpoints

- You stay in control by reviewing diffs and approving commands as they're proposed.
- Press Esc to interrupt mid-action and redirect; the agent stops and waits for new instructions.
- Permission prompts are natural checkpoints - read them rather than reflexively approving.
- Use plan mode for risky changes so the human reviews the strategy, not just the diff.
- The agent surfaces its reasoning and tool calls so you can audit what it did and why.

Hooks - automated guardrails in the loop

- Hooks run your shell commands at lifecycle events (e.g. `PreToolUse`, `PostToolUse`, `Stop`), independent of the model.
- Use them to enforce policy that shouldn't depend on the agent remembering - formatting, linting, blocking edits.
- A `PreToolUse` hook can deny an action by returning a non-zero exit, keeping a human-defined rule in the loop.

Hooks - automated guardrails in the loop (cont.)

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{ "type": "command", "command": "npx prettier --write \"$CLAUDE_FILE_PATHS\""}]
      }
    ]
  }
}
```

Hooks - automated guardrails in the loop (cont.)

- Hooks make cowork repeatable: the same checks fire no matter who is driving.

Delegating to subagents

- Subagents are separate Claude instances with their own context window, prompt, and tool permissions.
- The main session can hand a scoped task (e.g. "review this diff", "explore the auth module") to a subagent.
- Each subagent reports back a summary, keeping the main conversation focused and uncluttered.
- Define custom subagents (in `.claude/agents/`) with tailored instructions and restricted tools.
- Useful for parallel cowork: research, review, and implementation can run as distinct roles.

Sharing setup across a team

- Project-scoped `.claude/settings.json`, skills, hooks, and subagents can be committed to the repo.
- Plugins bundle skills, slash commands, agents, hooks, and MCP servers for one-step team distribution.
- MCP servers connect the agent to shared tools and data (issue trackers, databases, internal APIs).
- Committing this config means every teammate's sessions behave consistently - shared cowork conventions, not personal ones.
- `settings.local.json` stays per-user and out of version control for personal overrides.

Async collaboration and handoffs

- Long or background work can run detached and report back when it finishes, so you don't babysit it.
- Scheduled/triggered runs let the agent pick up recurring tasks and resume a session later.
- Write durable context (plans, decisions, TODOs) into files so a future session - or a teammate - can continue the work.
- A session is a handoff artifact: someone else can read the transcript and diffs to understand what happened.
- For programmatic cowork, the Agent SDK embeds the same agent loop into your own apps and pipelines.

Key takeaways

- Cowork is about tuning autonomy: plan mode and permission modes decide when the agent acts versus asks.
- Keep humans in the loop at meaningful checkpoints – review diffs, approve commands, interrupt with Esc.
- Encode shared context and guardrails in `CLAUDE.md`, `settings.json`, and hooks so collaboration is consistent and repeatable.
- Delegate scoped work to subagents and connect shared tools via MCP and plugins.
- Files and sessions are your handoff medium for async work and teammate collaboration.

14 · Claude Code Desktop & GitHub Workflow

How to drive Claude Code outside the terminal and use it to automate everyday GitHub work - pull requests, reviews, issues, and repo chores via the `gh` CLI.

Beyond the terminal

- Claude Code started as a terminal CLI but now also runs in a desktop/IDE context (VS Code and JetBrains extensions) and inside the Claude app, not just a raw shell.
- The IDE integration shares the same engine as the CLI: same `CLAUDE.md`, same `settings.json`, same permission model - only the front-end changes.
- The desktop/IDE surface adds inline diff viewing, file-tree awareness, and a panel UI, while still letting Claude run tools and edit files.
- You can run several sessions in parallel (e.g. one per worktree/branch) for independent tasks.
- Headless mode (`c\laude -p " ... "`) is the same tool with no UI - ideal for scripts, cron jobs, and CI.

One config, many surfaces

- `CLAUDE.md` (project root or `~/.claude/CLAUDE.md`) holds persistent context and conventions Claude reads automatically every session.
- `settings.json` controls permissions, env vars, hooks, and model choice; `.claude/settings.local.json` holds your personal, gitignored overrides.
- The same MCP servers, skills, subagents, and plugins are available whether you launch from terminal or IDE.
- Because config is file-based, switching front-ends does not change behavior - your guardrails travel with the repo.

One config, many surfaces (cont.)

```
// .claude/settings.json
{
  "permissions": {
    "allow": ["Bash(gh pr view:*)", "Bash(gh issue list:*)"],
    "deny": ["Bash(git push --force:*)"]
  }
}
```

The gh CLI is the bridge to GitHub

- Claude Code has no special GitHub API of its own; it shells out to the official `gh` CLI you already have authenticated (`gh auth login`).
- Anything you can do with `gh` - PRs, issues, reviews, releases, workflow runs - Claude can do by composing the right command.
- Keep `gh` calls under permission rules so read-only commands run freely while write actions stay gated.
- This keeps auth with GitHub, not with Claude: tokens live in your `gh` config, not in Claude's settings.

```
gh pr list --state open --limit 10
gh issue view 42 --comments
gh pr checks 128
```

Creating pull requests

- Ask Claude to branch, commit, and open a PR; it typically runs `git` for the commit and `gh pr create` for the PR.
- It can draft the PR title and body from the actual diff and recent commits, summarizing what changed and why.
- Commit/push only happens when you ask - Claude should not push to a default branch; have it branch first.
- House style (commit message format, required trailers, PR template) belongs in `CLAUDE.md` so every PR is consistent.

```
git checkout -b fix/login-redirect
git commit -am "fix: correct post-login redirect"
gh pr create --fill --base main
```

Reviewing pull requests

- Point Claude at a PR (`gh pr view`, `gh pr diff`) and ask for a focused review: correctness bugs, missing tests, risky changes.
- It can post findings as inline review comments via `gh pr review` / `gh api`, or just summarize in chat for you to act on.
- The built-in `/review` and `/security-review` commands target the current diff or a PR with a structured checklist.
- Use plan mode (read-only) for a review pass when you want analysis without any file edits or commands that change state.
- Tune depth explicitly - fewer high-confidence findings vs. broad coverage that may include uncertain ones.

Working with issues

- Claude can triage with `gh issue list` / `gh issue view`, then reproduce, propose a fix, and open a PR that closes the issue.
- Use closing keywords (`Closes #123`) in the PR body so GitHub links and auto-closes the issue on merge.
- It can label, comment, and assign via `gh issue edit` / `gh issue comment` when permissions allow.
- For large backlogs, subagents or skills can cluster duplicate reports by root cause before you plan fixes.
- Give Claude the issue number and let it gather context itself rather than pasting the whole thread.

```
gh issue create --title "Flaky auth test" --label bug
gh issue comment 42 --body "Reproduced on main; fix in #128"
```

Automating repo tasks

- Hooks in `settings.json` fire on events (e.g. `PostToolUse`, `Stop`) to run formatters, linters, or notifications deterministically - the harness runs them, not the model.
- Headless `claude -p` plus `gh` lets you script recurring chores: triage new issues, label PRs, draft release notes in CI.
- Subagents handle isolated sub-tasks (e.g. "review this file") with their own context, keeping the main thread clean.
- Skills and plugins package reusable workflows (PR babysitting, issue clustering, version bumps) you can invoke on demand.
- The Agent SDK lets you build your own programs on the same engine when a one-off command isn't enough.

Automating repo tasks (cont.)

```
// settings.json - auto-format after edits
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "prettier --write $CLAUDE_FILE_PATHS" }]
    }]
  }
}
```

Permissions & safety on GitHub

- Permission modes range from prompt-on-every-action to `acceptEdits` to fully autonomous - match the mode to the trust level of the task.
- Allowlist safe, read-only `gh` / `git` commands so they never interrupt you; gate or deny destructive ones (force push, branch delete).
- Plan mode is read-only: ideal for review and investigation where Claude should look but not touch.
- In CI/headless runs, prefer narrow, explicit allow rules over broad autonomy so an automated job can't run unexpected commands.
- Keep secrets out of `CLAUDE.md` and settings; let `gh` and the environment own authentication.

Key takeaways

- Same engine everywhere: terminal, IDE, and headless all share `CLAUDE.md`, `settings.json`, MCP, skills, and hooks.
- GitHub work flows through the authenticated `gh` CLI - PRs, reviews, and issues are just commands Claude composes.
- Encode your conventions in `CLAUDE.md` and your guardrails in `settings.json` so behavior is consistent and safe.
- Hooks, subagents, skills, and headless mode turn repetitive repo chores into reliable automation.
- Use plan mode and tight permission allowlists to review safely and to keep automated runs from doing anything destructive.

15 · Agent SDK

The Claude Agent SDK lets you run Claude Code's agent loop as a library inside your own programs, so you can build custom agents with the same tools, sessions, and permission controls that power the CLI.

What the Agent SDK is

- The same harness that powers the Claude Code CLI, exposed as a programmable library instead of a terminal app.
- Originally shipped as the "Claude Code SDK," later renamed the **Claude Agent SDK** to reflect that it builds general agents, not just coding tools.
- Available for **TypeScript/JavaScript** (`@anthropic-ai/claude-agent-sdk`) and **Python** (`claude-agent-sdk`).
- Gives you the full agent loop: model calls, tool execution, context management, and permission gating - all handled for you.
- Use it when you want Claude Code behavior embedded in an app, service, or batch job rather than driven interactively.

The agent loop as a library

- The core idea: you send a prompt, and the SDK runs the **loop** - Claude decides on actions, calls tools, reads results, and repeats until the task is done.
- You don't hand-write the orchestration; the SDK manages the turn-by-turn cycle of thinking, tool use, and responding.
- Results stream back as a sequence of **messages** (assistant text, tool calls, tool results, final result) you can consume as they arrive.
- The loop includes automatic **context compaction** so long-running tasks don't blow the context window.
- This is the same loop the CLI uses, so behavior is consistent between interactive sessions and programmatic ones.

Calling the SDK (TypeScript)

- The primary entry point is `query()`, which takes a prompt plus an `options` object and returns an async iterator of messages.

Calling the SDK (TypeScript) (cont.)

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const msg of query({
  prompt: "Find and fix the failing test in this repo",
  options: {
    permissionMode: "acceptEdits",
    allowedTools: ["Read", "Edit", "Bash"],
    cwd: "/path/to/project",
  },
})) {
  if (msg.type === "result") console.log(msg.result);
}
```

Calling the SDK (TypeScript) (cont.)

- `permissionMode`, `allowedTools`, `cwd`, `systemPrompt`, and `model` are common options.
- Authentication comes from `ANTHROPIC_API_KEY` (or a configured Bedrock/Vertex provider).

Calling the SDK (Python)

- Python offers a one-shot `query()` async generator and a stateful `ClaudeSDKClient` for multi-turn conversations.

Calling the SDK (Python) (cont.)

```
import anyio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for msg in query(
        prompt="Summarize the README in this folder",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob"],
            permission_mode="default",
        ),
    ):
        print(msg)

anyio.run(main)
```

Calling the SDK (Python) (cont.)

- Use `ClaudeSDKClient` when you need to keep a session open and send follow-up turns interactively.

Tools available to your agent

- Built-in Claude Code tools are available: `Read`, `Write`, `Edit`, `Bash`, `Glob`, `Grep`, `WebFetch`, `WebSearch`, `TodoWrite`, and the `Task` subagent tool.
- Control exposure with `allowedTools` / `disallowedTools` so an agent only gets the capabilities it needs.
- Add your own tools in two ways: connect **MCP servers**, or define **in-process custom tools** (e.g. `createSdkMcpServer` + `tool()` in TS) that run inside your application.
- In-process tools are great for app-specific actions (query your DB, call your internal API) without spawning a separate process.
- Tool results feed back into the loop automatically; you generally don't manually wire results into the next prompt.

Sessions and conversation state

- A **session** is one continuous conversation with its accumulated context, history, and working directory.
- The TS `query()` returns a session you can resume; Python's `ClaudeSDKClient` keeps the session alive for back-and-forth turns.
- Sessions can be **resumed** by ID so an agent can pick up where it left off across separate runs.
- Long sessions are kept manageable through automatic compaction of older context.
- Each session has its own permission state and working directory, so concurrent agents stay isolated.

Permissions and safety controls

- The same **permission modes** as the CLI apply: `default` (ask), `acceptEdits` (auto-approve file edits), `plan` (read-only planning, no mutations), and `bypassPermissions` (skip prompts - use with care).
- Fine-grained control comes from `allowedTools` / `disallowedTools` and a programmatic `canUseTool` callback that approves or denies each tool call.
- The `canUseTool` hook lets you implement custom policy - log requests, prompt a human, or block dangerous commands.
- **Hooks** (`PreToolUse`, `PostToolUse`, etc.) can run your code at lifecycle points to validate, modify, or veto actions.
- Because the SDK can act autonomously, lock down permissions deliberately - `bypassPermissions` removes the safety net.

Reusing Claude Code configuration

- The SDK can load familiar Claude Code config: **CLAUDE.md** memory files, **settings.json**, **subagents**, **skills**, and **MCP servers**.
- A `CLAUDE.md` in the working directory supplies project context, just like in the CLI.

```
# Project: billing-service
- Run tests with `pytest -q`
- Never edit files under `migrations/` directly
- Prefer async functions for new I/O code
```

- You can also set the `systemPrompt` option directly, or extend the default Claude Code system prompt rather than replacing it.
- **Subagents** let the main agent delegate scoped work via the `Task` tool, keeping the primary context clean.
- **Skills** and **plugins** can extend the agent with reusable instructions and bundled tooling.

Where the SDK fits

- Good fits: CI bots that fix or review code, batch refactors across many repos, automated triage, custom internal "agent" products, and chat backends.
- It abstracts away prompt construction, tool wiring, retries, and context management so you focus on the task and policy.
- Streaming output makes it suitable for live UIs that show the agent's progress turn by turn.
- For headless/non-interactive runs, choose a permission mode that needs no human prompts (and accept the added risk).
- It's the supported path for embedding Claude Code capabilities - more robust than scripting the CLI binary directly.

Key takeaways

- The Agent SDK is Claude Code's agent loop packaged as a TypeScript and Python library - same tools, sessions, and permissions, programmatically.
- `query()` runs the loop and streams messages; `ClaudeSDKClient` (Python) keeps multi-turn sessions alive and resumable.
- You extend agents with MCP servers and in-process custom tools, and constrain them with `allowedTools`, permission modes, `canUseTool`, and hooks.
- It reuses `CLAUDE.md`, `settings.json`, subagents, and skills, so config carries over from the CLI.
- Set permissions deliberately for autonomous runs - convenience modes like `bypassPermissions` trade safety for fewer prompts.

16 · Wrapping Up

How Claude Code's building blocks - CLAUDE.md, permissions, skills, hooks, MCP, subagents, and plugins - fit together into one customizable agent, plus a workflow to use them well.

The big picture

- Claude Code is an agentic CLI: a model that reads, edits, and runs code in your terminal using tools.
- Everything else is configuration that shapes *what context it has, what it's allowed to do, and how it behaves*.
- Three broad layers: **context** (CLAUDE.md, MCP), **control** (permissions, hooks), and **capability** (skills, subagents, plugins).
- Almost all of it lives in plain files (`CLAUDE.md`, `.claude/settings.json`, `.claude/...`) that you can commit and share.
- Start minimal; add pieces only when a real friction point shows up.

CLAUDE.md - persistent context

- Auto-loaded memory file that gives Claude project conventions, commands, and gotchas every session.
- Lives at several levels: project root `CLAUDE.md` (shared), `CLAUDE.local.md` or `.claude/` for personal notes, and `~/ .claude/CLAUDE.md` for global preferences.
- Keep it short and high-signal - it's spent on every turn, so bloat costs context.
- Use `@path/to/file.md` imports to pull in modular docs instead of one giant file.

```
# Project: payments-api
- Run tests: `pnpm test`
- Never edit files in `generated/` - they're built from protobufs.
- Use the `Result<T>` type for fallible functions, not exceptions.
```

Permissions - what Claude may do

- Tool calls are gated by allow / ask / deny rules in `settings.json`; sensitive actions prompt unless pre-approved.
- Rules are specific, e.g. `Bash(npm run test:*)` or `Read(./src/**)`, and resolve deny > ask > allow.
- Permission modes change the default posture: `default`, `acceptEdits`, `plan` (read-only investigation), and `bypassPermissions` (no prompts - use with care).
- Settings cascade: enterprise > project (`.claude/settings.json`) > user (`~/.claude/settings.json`), with `.local.json` for uncommitted overrides.

Permissions - what Claude may do (cont.)

```
{  
  "permissions": {  
    "allow": ["Bash(git status)", "Read(./**)"],  
    "ask": ["Bash(git push:*)"],  
    "deny": ["Read(./env)", "Bash(rm -rf:*)"]  
  }  
}
```

Skills - packaged know-how

- A skill is a folder with a `SKILL.md` describing a repeatable procedure, plus optional scripts and reference files.
- Claude reads the name/description up front and loads the full body only when the task matches - progressive disclosure keeps context lean.
- Great for codifying workflows your team repeats: releasing, writing a certain kind of test, generating a report.
- Invoked automatically when relevant, or explicitly as a slash command; they can ship inside plugins.

```
---  
  
name: release-notes  
description: Generate release notes from merged PRs since the last tag.  
  
---
```

```
1. Find the last git tag.
```

Hooks - deterministic automation

- Shell commands the harness (not the model) runs on lifecycle events: `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `Stop`, `SessionStart`, and more.
- Use them for guarantees the model shouldn't be trusted to remember - auto-formatting after edits, blocking writes to protected paths, logging.
- A `PreToolUse` hook can return a non-zero/deny decision to **block** an action before it happens.
- Configured in `settings.json`; they fire every time, which is exactly why they're reliable.

Hooks - deterministic automation (cont.)

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "prettier --write \"$CLAUDE_FILE_PATHS\"" }]
    }]
  }
}
```

MCP - external tools and data

- Model Context Protocol connects Claude to outside systems: databases, issue trackers, browsers, internal APIs.
- MCP servers expose **tools** (actions) and **resources** (readable context) over a standard interface, so any compliant server just works.
- Add servers per-project or globally; their tools appear with an `mcp__<server>__<tool>` naming pattern and obey the same permission rules.
- This is how Claude Code reaches beyond your filesystem and shell without baking integrations into the core tool.

Subagents - delegated context

- Subagents are separate Claude instances Claude can spawn for a focused task, each with its own context window.
- Keeps the main thread clean: a subagent can read 50 files or run a noisy search and return just the answer.
- Configurable with their own system prompt, tool allowlist, and model - define reusable ones in `.claude/agents/`.
- Good for parallelizable or exploratory work (codebase search, multi-file review); they can't see each other, so give each a self-contained brief.

Plugins - bundling and sharing

- A plugin packages skills, slash commands, hooks, subagents, and MCP server configs into one installable unit.
- Distributed via marketplaces (git repos with a manifest), so a team can install a shared toolkit with one command.
- This is the unit of *distribution* - it ties together the other primitives so onboarding a repo means installing one thing.
- Use it to standardize how your whole team works with Claude Code instead of everyone hand-configuring.

How the pieces fit together

- **CLAUDE.md** tells Claude about your project; **MCP** gives it access to systems beyond the repo.
- **Permissions** and **hooks** form the guardrails – one decides what's allowed, the other enforces guarantees deterministically.
- **Skills** and **subagents** extend capability – reusable procedures and isolated worker contexts.
- **Plugins** bundle all of the above so it's shareable and consistent across a team.
- The model is constant; these layers turn a generic agent into one that fits *your* codebase.

Recommended workflow

- **Explore → Plan → Implement → Verify**: understand first, agree on an approach, then change code, then check it.
- Use **plan mode** for anything non-trivial so Claude investigates read-only before touching files.
- Keep changes reviewable: small commits, run tests/linters (ideally enforced via a hook), read the diff yourself.
- Manage context: start fresh sessions for new tasks, use `/clear`, and push heavy investigation into subagents.
- Capture recurring fixes and conventions back into CLAUDE.md and skills so the setup compounds over time.

Where to go next

- Read the official docs at `docs.claude.com` (Claude Code section) for the authoritative, up-to-date reference.
- Use `/help` in-session and `claude --help` for current commands and flags.
- For programmatic / headless use, explore the **Agent SDK**, which exposes the same agent loop for building your own tools and automations.
- Browse community plugins and marketplaces, but vet hooks and MCP servers before trusting them – they run with your permissions.

Key takeaways

- Claude Code = one model plus configurable layers for context, control, and capability - almost all just files you commit.
- Permissions and hooks keep the agent safe and predictable; CLAUDE.md and MCP keep it well-informed.
- Skills, subagents, and plugins let you scale and share customizations across tasks and teammates.
- A disciplined explore-plan-implement-verify loop beats one-shot prompting on real work.
- Start small, add a piece only to solve a real pain point, and feed lessons back into your config.